

Advanced Artificial Intelligence

Next-Word Prediction Engine
Using Markov Chain / N-gram NLP Model

Vo Chau Anh Kiet
Nguyen Quoc Bao
Vo Thanh Phi Long

Ho Chi Minh City Open University

April 27, 2026

Contents

1 Introduction

1.1 Problem Statement

Predicting the next word in a sequence is a fundamental challenge in Natural Language Processing (NLP). Applications range from smartphone keyboards and autocomplete systems to search engines and accessibility tools. The core question is: given a sequence of words as evidence, what is the most probable next word?

1.2 Motivation

This project applies the Markov Chain model — a foundational probabilistic framework — to construct an *intelligent system* capable of next-word prediction. The Markov property states that the future state depends only on the current state (or a limited history), which directly translates to the n-gram assumption in NLP.

1.3 Project Goals

- Build a probabilistic language model using bigram and trigram Markov Chains.
- Design a Knowledge Base (n-gram frequency table) and an Inference Engine (transition probability calculator).
- Wrap the model in a FastAPI web service.
- Provide an interactive React user interface for demonstration.

2 Related Concepts

2.1 Markov Chains

A *Markov Chain* is a stochastic process satisfying the Markov property:

$$P(X_{n+1} \mid X_n, X_{n-1}, \dots, X_1) = P(X_{n+1} \mid X_n)$$

In the context of language, each state is a word (or sequence of words), and the transition probability represents the likelihood of one word following another.

2.2 N-gram Language Models

An *n-gram* is a contiguous sequence of n items from a text. For next-word prediction:

- **Bigram** ($n = 2$): Predicts w_i based on w_{i-1} .

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

- **Trigram** ($n = 3$): Predicts w_i based on (w_{i-2}, w_{i-1}) .

$$P(w_i \mid w_{i-2}, w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

2.3 Laplace (Add-1) Smoothing

To handle unseen n-grams (zero-probability problem), Laplace smoothing adds a count of 1 to every possible n-gram:

$$P_{\text{smooth}}(w \mid \text{context}) = \frac{C(\text{context}, w) + 1}{C(\text{context}) + |V|}$$

where $|V|$ is the vocabulary size.

3 System Architecture

3.1 High-Level Overview

The system follows a three-layer architecture:

1. **Frontend (React)**: Accepts user text as evidence; displays top- k predictions with probabilities.
2. **Backend (FastAPI)**: Exposes a REST endpoint `POST /predict`; delegates to the AI model layer.
3. **AI Model Layer**: Contains the Knowledge Base and Inference Engine.

The data flow is:

User Input $\xrightarrow{\text{HTTP POST}}$ FastAPI $\xrightarrow{\text{tokenise}}$ Preprocessor $\xrightarrow{\text{context}}$ Inference Engine $\xrightarrow{\text{JSON}}$ React UI

3.2 Project Structure

```
markov-nlp/
├── backend/
│   ├── main.py           # FastAPI app + CORS + routes
│   └── model/
│       ├── markov_model.py # KB + Inference Engine
│       ├── services/
│       │   ├── predictor.py # Orchestration service
│       │   └── schemas/
│       │       ├── predict.py # Pydantic request/response
│       └── data/
│           └── corpus.txt    # Training corpus
└── frontend/
    └── src/
        ├── App.jsx        # Main React component
        └── components/    # PredictionBar, StatsPanel
```

4 Knowledge Base & Inference Engine Design

4.1 Knowledge Base: N-gram Frequency Table

The Knowledge Base is implemented as a nested dictionary:

$$\text{ng_counts}[(\text{context})] = \text{Counter}\{w_1 : c_1, w_2 : c_2, \dots\}$$

Example — Given the sentence *"the cat sat on the mat"*:

Context	Next words (counts)
(the)	{cat: 1, mat: 1}
(cat)	{sat: 1}
(sat)	{on: 1}
(on)	{the: 1}

4.2 Inference Engine: Transition Probabilities

Given user input *"the cat"*, the engine extracts the context ("cat",) and computes:

$$P(\text{sat} \mid \text{cat}) = \frac{C(\text{cat}, \text{sat}) + 1}{C(\text{cat}) + |V|} = \frac{1 + 1}{1 + 50} \approx 0.039$$

The engine returns the top- k words sorted by descending probability.

5 Implementation Details

5.1 Text Preprocessing

```
def preprocess(text: str) -> List[str]:
    text = text.lower()
    text = re.sub(r"[^a-z\s]", "_", text)
    return [t for t in text.split() if t]
```

5.2 Building the N-gram Knowledge Base

```
for i in range(len(tokens) - context_size):
    context = tuple(tokens[i : i + context_size])
    next_word = tokens[i + context_size]
    self.ng_count[context][next_word] += 1
    self.ctx_total[context] += 1
```

5.3 Markov Inference with Laplace Smoothing

```
prob = (count + 1) / (ctx_total + self.V)
```

5.4 FastAPI Endpoint

```
@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    results = predictor.predict(
        text=request.text, top_k=request.top_k, n=request.n
    )
    return PredictResponse(input_text=request.text,
                           predictions=results,
                           model_order=request.n)
```

6 Experimental Results

6.1 Test Cases

Table 1: Bigram Model Predictions (Top-3)

Input Text	Predicted #1	Predicted #2	Predicted #3
"the"	cat	dog	sun
"machine"	learning	learning	—
"natural"	language	world	—
"the cat"	sat	—	—

6.2 Model Statistics

Table 2: Knowledge Base Statistics

Metric	Value
Total tokens in corpus	$\approx 1,000$
Unique vocabulary size	≈ 300
Bigram contexts	≈ 250
Trigram contexts	≈ 400